

Practical: 5

Implementation of a knapsack problem using dynamic programming.

25/08/2026

Aim: To Implement the knapsack problem using dynamic programming.

Algorithm:

Algorithm:-
Input: number of items n , capacity w , weight and profit of each item.
Output: Maximum profit.
Step 1: Start
Step 2: Read n, w , weight[] and profit[]
Step 3: Create a DP table $K[n+1][w+1]$
Step 4: Initialize the first row and first column of the table to 0.
Step 5: for $i = 1$ to n , do:
 for $w = 1$ to w , do:
 if weight $[i-1] \leq w$, then
 include = profit $[i-1] + K[i-1][w - \text{weight}[i-1]]$
 exclude = $K[i-1][w]$
 $K[i][w] = \max(\text{include}, \text{exclude})$
Step 6: if weight $[i-1] > w$, then item cannot be included
 $K[i][w] = K[i-1][w]$
Step 7: ~~if weight $[i-1] > w$~~
Step 7: Repeat step 5 and step 6 for all items and capacities
Step 8: After completing the table, the maximum profit is stored at:
 $K[n][w]$
Step 9: Display $K[n][w]$ as the maximum profit
Step 10: Stop

Program:

```
#include<iostream>
using namespace std;

class Knapsack {

private:
    int w[10];    // weights
    int v[10];    // values
```

```
int n;          // number of items
int W;          // capacity
int dp[10][100]; // dp table
```

public:

```
// ?? Get Input ??
void getInput() {
    cout << "Enter number of items  : ";
    cin >> n;
    cout << "Enter knapsack capacity : ";
    cin >> W;
    for(int i = 1; i <= n; i++) {
        cout << "Item " << i << " weight and value : ";
        cin >> w[i] >> v[i];
    }
}
```

// ?? Build dp table ??

```
void solve() {

    // base case - row 0 and col 0
    for(int i = 0; i <= n; i++) dp[i][0] = 0;
    for(int j = 0; j <= W; j++) dp[0][j] = 0;

    // fill table
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= W; j++) {

            if(w[i] <= j) {
                // take or not take
                dp[i][j] = max(
                    v[i] + dp[i-1][j-w[i]],
                    dp[i-1][j]
                );
            } else {
                // cannot take item i
                dp[i][j] = dp[i-1][j];
            }
        }
    }
}
```

// ?? Print dp table ??

```
void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "  ";
```

```
for(int j = 0; j <= W; j++)  
    cout << j << " ";  
cout << endl;
```

```
for(int i = 0; i <= n; i++) {  
    cout << "i=" << i << " ";  
    for(int j = 0; j <= W; j++)  
        cout << dp[i][j] << " ";  
    cout << endl;  
}  
}
```

```
// ?? Find selected items ??
```

```
void findItems() {  
    cout << "\nSelected Items:" << endl;  
    int j = W;  
    for(int i = n; i >= 1; i--) {  
        if(dp[i][j] != dp[i-1][j]) {  
            cout << "Item " << i  
                << " (w=" << w[i]  
                << ", v=" << v[i]  
                << ") TAKEN " << endl;  
            j = j - w[i];  
        } else {  
            cout << "Item " << i  
                << " (w=" << w[i]  
                << ", v=" << v[i]  
                << ") NOT taken " << endl;  
        }  
    }  
}
```

```
// ?? Show result ??
```

```
void showResult() {  
    cout << "\nMaximum Value = "  
        << dp[n][W] << endl;  
}  
};
```

```
// ?? MAIN ??
```

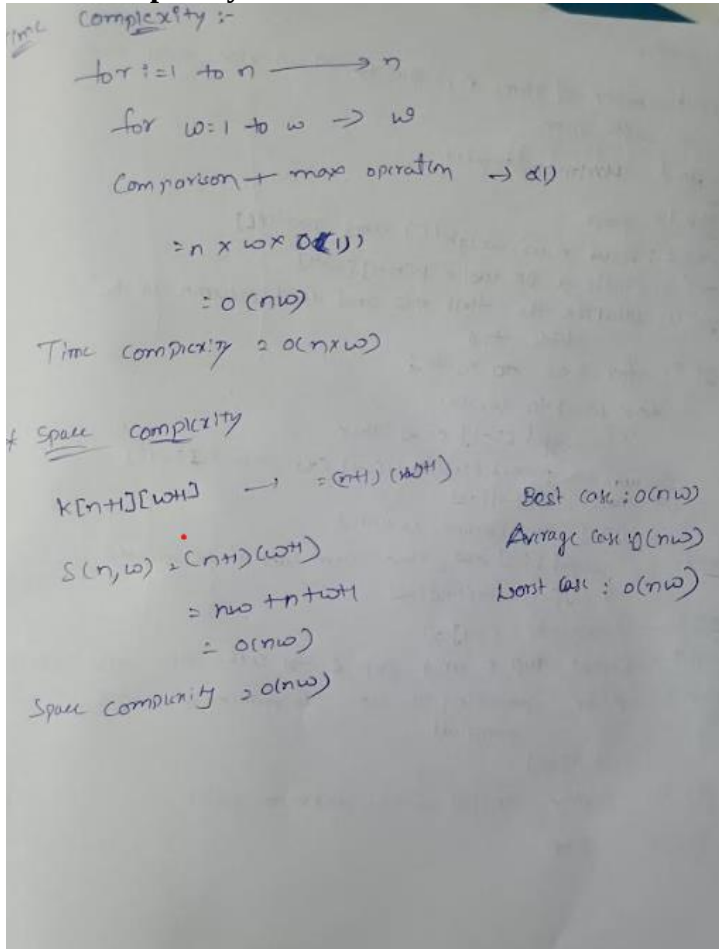
```
int main() {  
  
    Knapsack k;  
  
    k.getInput();  
    k.solve();  
    k.printTable();
```

```
k.showResult();  
k.findItems();  
  
return 0;  
}
```

Output:

```
Enter number of items : 2  
Enter knapsack capacity : 6  
Item 1 weight and value : 2 6  
Item 2 weight and value : 6 8  
  
DP Table:  
    0  1  2  3  4  5  6  
i=0  0  0  0  0  0  0  0  
i=1  0  0  6  6  6  6  6  
i=2  0  0  6  6  6  6  8  
  
Maximum Value = 8  
  
Selected Items:  
Item 2 (w=6, v=8) TAKEN  
Item 1 (w=2, v=6) NOT taken  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Time Complexity:



Conclusion:

The 0/1 Knapsack problem was successfully implemented using the Dynamic Programming technique. The algorithm determines the maximum profit that can be obtained by selecting items within the given knapsack capacity. The time complexity is $O(nW)$ and the space complexity is $O(nW)$. Thus, dynamic programming provides an efficient solution compared to checking all possible combinations.